AGENTIC AI · LEARNING NOTES

A2A Protocol
Agent-to-Agent

Complete notes from the live class, the handwritten board and the study material — plus the class code split into **3 hands-on practical classes** you can run on Windows.

MCP recap | A2A theory | JSON-RPC · gRPC · HTTP+JSON | Python a2a-sdk | Interview Q&A

01
Build an A2A Server
Flight Agent, Agent Card, raw JSON-RPC call

02
Build an A2A Client
Discover, read skills, SendMessage

03
Multi-Agent Orchestrator
Flight + Hotel + Visa, parallel delegation

Class session: 24 Oct 2026 · Instructor: Sunny

Code tested with a2a-sdk 1.3.0 · Python 3.13

Contents

PART A — THEORY

A1 Recap: AI systems, context & MCP — why MCP exists, client/server, STDIO vs HTTP

A2 What is A2A and why was it introduced? — the custom-integration problem

A3 The 4 phases of AI systems — LLM → tools → MCP → multi-agent

A4 When do you need A2A (and when not)? — Research & Writer agent example

A5 MCP vs A2A — full architecture + comparison table

A6 Protocol bindings — JSON-RPC, gRPC, HTTP+JSON

A7 Core A2A terms — Card, Skill, Message, Task, Artifact...

A8 Python SDK classes — AgentExecutor, EventQueue, RequestHandler...

PART B — PRACTICAL CLASSES

B0 Setup on Windows — folders, venv, install

01 Class 01 — Your first A2A Server (Flight Agent)

02 Class 02 — Your first A2A Client

03 Class 03 — Multi-Agent Travel Orchestrator

PART C — REVISION

C1 Interview Q&A (from the doubt session)

C2 One-page cheat sheet

HOW TO USE THESE NOTES

Read Part A once to get the big picture. Then do the three classes **in order** — each one reuses what you built before. Every code block has a coloured label: **FOR CLASS_01**, **FOR CLASS_02**, **FOR CLASS_03** and shows the exact file name to create.

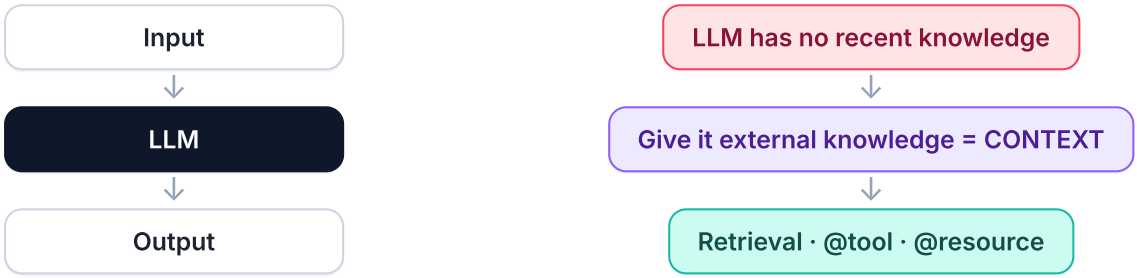
PART A

Theory — From MCP to A2A

Everything explained on the board in class, cleaned up and in order.

A1 Recap: AI systems, context & MCP

Every AI system works the same basic way: we give **input** to an **LLM** and get an **output**. The problem: the LLM does **not have recent knowledge** (or your private data). So we give it **external knowledge**, also called **context**.



Context can come from **retrieval** (a vector database), **tools** (API calls) and **resources** (files, data). You can connect all of this **without MCP** too. What MCP adds is a **standard way** to do it.

MCP = Model Context Protocol	Meaning
Model	The LLM
Context	The outside knowledge we give it (tools, resources, prompts)
Protocol	A standard set of rules → **client / server** architecture
Released by	Anthropic (2024)

MCP building blocks

- **Client** = your AI application (e.g. the travel agent app).
- **Server** = hosts the context: `@tool`, `@resource`, `@prompt`.
- **Transport**: **STDIO** (same machine, good for practice) or **HTTP(S)** (over a network — what real projects use; the URL can be localhost, AWS, Azure...).

The class MCP example: Business Trip Co-pilot

One **client** (`app_http.py`, the travel agent) talks to **3 MCP servers** over HTTP:

Server	What it hosts
Policy server	Company travel policy in a vector DB + a **retriever exposed as a tool**
Travel server	Flight / hotel search tools (dummy data; can call a real API)
Work server	Trip task tools — e.g. create trip task, list trip tasks

To run it: first start the servers (`python start_servers.py` launches all three), then in a **second** terminal run the client (`python app_http.py`). The app asked for destination, days and origin city, built a full trip

plan, and asked for **human approval** before booking the hotel.

> **INSTRUCTOR TIP**
> Use **Command Prompt (cmd)** instead of PowerShell for these demos. Always start the **server first**, then the client.

A2 What is A2A and why was it introduced?

A2A = Agent-to-Agent Protocol · published by Google in 2025

The problem: different agentic systems are built with different frameworks. If Agent A must talk to Agent B, you write **custom integration** code. Add Agent C (maybe an external company's agent) → another custom integration. With many agents this becomes a mess.

Without A2A



...and a new custom integration for Agent C, D, E...

With A2A



Every compatible agent speaks the same language

Imagine 100 companies built 100 agents, each with its own request format, response format and API style. A2A solves this **interoperability** problem: a common set of rules so compatible agents can discover each other, delegate tasks, share status and return results.

A2A is framework-independent. It does not matter which framework built the agent:

LangGraph	Google ADK	OpenAI Agents SDK	CrewAI	AutoGen
✓	✓	✓	✓	✓

> **ASAAN ALFAAZ MEIN**
> MCP ne **tool calling** ko standard banaya. A2A **agents ki aapas ki baat-cheet** (delegation) ko standard banata hai. Do alag frameworks ke agents bhi ek hi zubaan mein baat kar sakte hain.

A3 The 4 phases of AI systems

PHASE 1

`LLM → Answer`

Plain model answers from its training data.

PHASE 2

`LLM → Tools → API/DB/Files`

Tools are basically API calls.

PHASE 3

`LLM → MCP → Tools/Data`

Tool access is standardized with MCP.

PHASE 4

`Agent ↔ A2A ↔ Agent`

Many agents (Travel, Finance, HR) must discover each other, delegate tasks, share status and return results.

Board example for phase 4: a **User** talks to an **Employee Service Agent**, which uses A2A to reach the **HR Agent**, **IT Agent** and **Finance Agent**.

A4 When do you need A2A (and when not)?

Example: **Agent A** does research, **Agent B** writes articles. Both are built with LangGraph.

Situation 1 — same application

Both agents live in the same Python app, same server. Just pass the output along (a **handoff**):

```
research = research_agent.invoke({"topic": "AI"})
article = writer_agent.invoke({"research": research})
```

A2A is NOT required.

Situation 2 — different servers

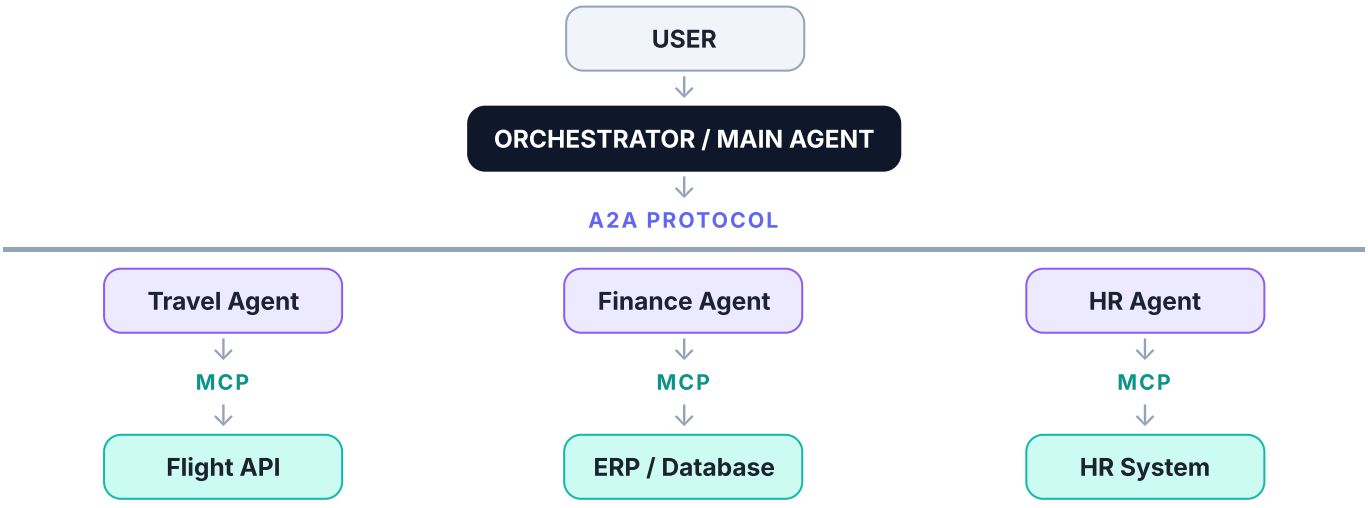
The Writer Agent runs on your friend's server. `writer_agent.invoke()` will **not** work — it is not a local object.

Option 1: a normal REST API (`requests.post(".../write")`) — works, but every company invents its own format.
Option 2: expose it as an **A2A Server**, call it with an **A2A Client** — one standard for everyone.

IMPORTANT

A2A is about **design**, not output. You can get the same result without A2A. Use it when it makes the communication between agents cleaner and standard — don't force it into every project.

A5 MCP vs A2A

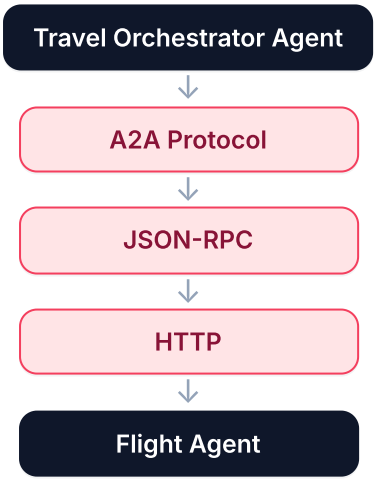


A2A connects agents to agents. MCP connects those agents to tools and data.

	MCP	A2A
Full form	Model Context Protocol	Agent-to-Agent Protocol
Released by	Anthropic (2024)	Google (2025)
Connects	Agent → tools, databases, APIs, files	Agent → other agents (multi-agent system)
Standardizes	How context reaches the model	How agents communicate & delegate tasks
Architecture	Client / Server	Client / Server
Server exposes	@tool, @resource, @prompt	Agent Card + Skills + message endpoint
How often used	Very common in real projects	Less common; used when agents are separate services

A6 Protocol bindings: JSON-RPC, gRPC, HTTP+JSON

A2A = the communication rules. **Protocol binding** = the mechanism that carries those rules over the network. A2A has **3 standard bindings** (custom ones like WebSocket or MQTT are also possible).



Feature	JSON-RPC	gRPC	HTTP+JSON
Request made by	Method name **inside** the JSON	Remote function call	HTTP method + URL
Data format	JSON	Protocol Buffers (binary)	JSON
Transport	HTTP(S)	HTTP/2	HTTP(S)
Example	`"method": "SendMessage"`	`SendMessage()`	`POST /message:send`

RPC = Remote Procedure Call (calling a function that runs on another machine). **JSON-RPC** = JavaScript Object Notation Remote Procedure Call. Our class code uses **JSON-RPC**. A JSON-RPC request looks like this:

```
{
  "jsonrpc": "2.0",
  "method": "SendMessage",
  "params": {
    "message": {
      "role": "ROLE_USER",
      "parts": [{"text": "Find flights to Dubai"}]
    }
  },
  "id": 1
}
```

ASAAN ALFAAZ MEIN

JSON-RPC aur HTTP+JSON dono HTTP aur JSON use karte hain. Farq sirf itna hai: JSON-RPC mein **operation ka naam JSON ke andar** hota hai, HTTP+JSON mein operation **URL aur HTTP method** se pata chalta hai. Class 01 mein aap khud raw JSON-RPC bhejoge.

A7 Core A2A terms

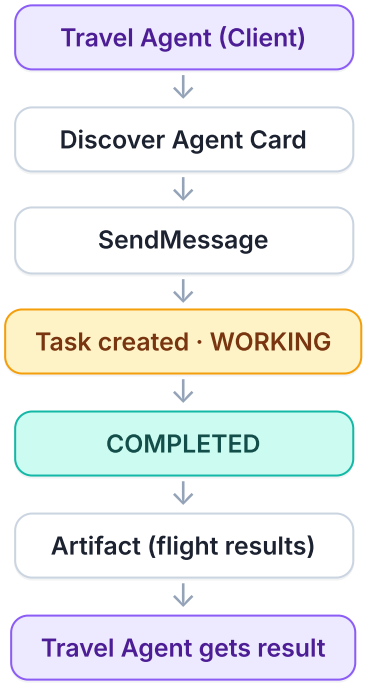
Example used for every term: **Travel Agent → A2A → Flight Agent**, user asks “Find flights from Bengaluru to Dubai.”

#	Term	Simple meaning	Flight example
1	**A2A Client**	The agent/app that **sends** a request	Travel Agent
2	**A2A Server**	The agent that **receives** requests and does the work	Flight Agent
3	**Agent Card**	A digital business card (JSON) about the agent	Name, URL, skills, security
4	**Agent Skill**	What the agent can do	Search Flights
5	**Agent Discovery**	Finding an agent by reading its card	Travel Agent finds Flight Agent
6	**Message**	Information exchanged between agents	“Find flights to Dubai”
7	**Part**	The actual content inside a message	Text, file or JSON data
8	**Task**	A trackable unit of work	Flight search task
9	**Task ID**	Unique id of a task	task-101
10	**Task Status**	Current state of a task	Working, Completed, Failed
11	**Artifact**	Output/deliverable produced by a task	Structured list of flights
12	**Context ID**	Groups related messages and tasks	context-501

Advanced term	Meaning
Agent Interface	Endpoint URL + protocol binding + protocol version
Agent Capabilities	Features supported, e.g. streaming, push notifications
Authentication	Who is allowed to call the agent
SendMessage / GetTask	Operations: send a message / get a task's status or result
Streaming (SSE) / Polling	Live updates while working / repeatedly checking if done
Push Notification	Server sends task updates to your webhook automatically
Task Lifecycle	Submitted → Working → Completed (or Failed)

MESSAGE VS TASK VS ARTIFACT

Message = an instruction or reply. **Task** = a trackable job with an ID and status. **Artifact** = the final deliverable. Not every message creates a Task — our class agents reply with a **direct Message**, which is the simplest flow.



A8 Python SDK classes you will use

Class	Purpose	Used in
`AgentCard`	The agent's info & metadata	Server
`AgentSkill`	Describes one skill of the agent	Server
`AgentInterface`	Endpoint URL + binding (JSONRPC) + version	Server
`AgentExecutor`	Bridge between an A2A request and **your** agent logic (`execute()`)	Server
`RequestContext`	Info about the incoming request (e.g. `context.message`)	Server
`EventQueue`	Where the agent puts its reply / status events	Server
`DefaultRequestHandler`	Handles incoming A2A requests and calls the executor	Server
`InMemoryTaskStore`	Stores tasks and their states (in RAM)	Server
`Starlette` + `uvicorn`	Web app that routes requests / runs the server	Server
`A2ACardResolver`	Downloads another agent's Agent Card (discovery)	Client
`create_client` , `ClientConfig`	Builds an A2A client from a card	Client
`SendMessageRequest`	Wraps the message you send	Client

Recommended learning order: **Client → Server → Agent Card → Agent Skill → Message → Task → Artifact**. After that, AgentExecutor, RequestHandler, EventQueue and the bindings become easy.

PART B

Practical Classes — Setup

Do this once before Class 01. All commands are for Windows Command Prompt.

B0 Folder structure

Project layout

```
a2a_classes\
|-- requirements.txt
|-- class_01\
|     |-- flight_agent.py        <- A2A server (port 9999)
|     |-- check_card.py          <- read the Agent Card
|     `-- raw_jsonrpc_call.py    <- send raw JSON-RPC by hand
|-- class_02\
|     |-- flight_client.py       <- A2A client (discover + send)
|     `-- server_down_test.py    <- what if the agent is offline?
`-- class_03\
      |-- hotel_agent.py         <- A2A server (port 10000)
      |-- visa_agent.py          <- A2A server (port 10001, challenge)
      `-- travel_orchestrator.py <- calls all agents in parallel
```

Install (one time)

Command Prompt (cmd)

```
cd a2a_classes
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

ALL CLASSES **requirements.txt**

```
a2a-sdk>=1.3.0
uvicorn
starlette
httpx
```

PACKAGE NAME

Install `a2a-sdk` (the A2A Python SDK). You do **not** need Google ADK to use A2A. The code was tested with **a2a-sdk 1.3.0**; older 0.x versions use different import paths.

EVERY NEW TERMINAL

Each server needs its own Command Prompt window. In every new window run `cd a2a_classes` then `venv\Scripts\activate` before starting a file.

PRACTICAL CLASS 01

Your first A2A Server — the Flight Agent

Build an agent that other agents can discover and talk to.

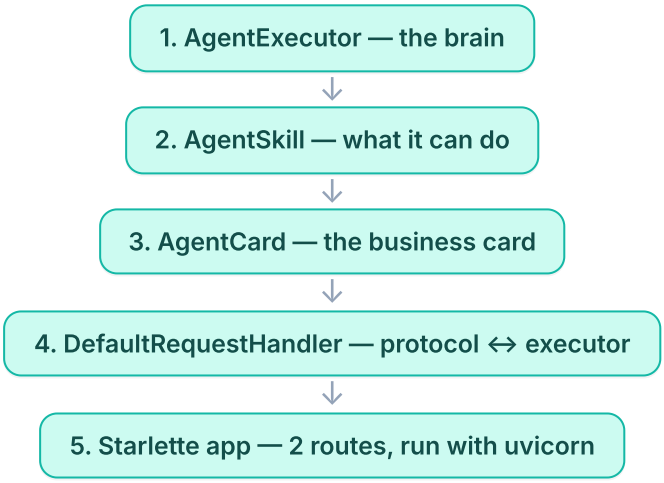
Goal

- Run an A2A server on port 9999
- See its Agent Card in the browser
- Send it a raw JSON-RPC message by hand and see what travels on the wire

Concepts

- A2A Server, Agent Card, Agent Skill
- AgentExecutor, RequestContext, EventQueue
- DefaultRequestHandler, Starlette routes
- JSON-RPC binding

The 5 building blocks of every A2A server



FOR CLASS_01 `class_01/flight_agent.py` Server · port 9999

```python
# ============================================================
# CLASS 01 - flight_agent.py
# Your first A2A SERVER: the Flight Agent
# Run:  python flight_agent.py      (keep this terminal open)
# ============================================================
import uvicorn

from a2a.helpers import get_message_text, new_text_message
from a2a.server.agent_execution import AgentExecutor, RequestContext
from a2a.server.events import EventQueue
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.routes import create_agent_card_routes, create_jsonrpc_routes
from a2a.server.tasks import InMemoryTaskStore
from a2a.types import AgentCapabilities, AgentCard, AgentInterface, AgentSkill
from starlette.applications import Starlette


# ---------- STEP 1: The "brain" (AgentExecutor) ----------
# A2A calls execute() every time a message arrives.
class FlightAgentExecutor(AgentExecutor):
    async def execute(self, context: RequestContext, event_queue: EventQueue) -> None:
        # context.message = the message the client sent us
```

```
        query = get_message_text(context.message) if context.message else ""

        # Dummy logic (later you can call a real flight API or an LLM here)
        result = (
            "Flight Agent result:\n"
            "Bangalore -> Dubai\n"
            "1. Emirates EK569 | Non-stop | ₹28,000\n"
            "2. Air India AI995 | 1 stop | ₹24,500\n"
            f"\nReceived task: {query}"
        )

        # Put the reply in the EventQueue -> A2A sends it back to the client
        await event_queue.enqueue_event(new_text_message(result))

    async def cancel(self, context: RequestContext, event_queue: EventQueue) -> None:
        raise NotImplementedError("Cancellation is not supported.")


# ---------- STEP 2: What this agent can do (AgentSkill) ----------
skill = AgentSkill(
    id="search_flights",
    name="Search Flights",
    description="Searches flight options between cities.",
    input_modes=["text/plain"],
    output_modes=["text/plain"],
    tags=["travel", "flight"],
    examples=["Find flights from Bangalore to Dubai"],
)

# ---------- STEP 3: The agent's "business card" (AgentCard) ----------
agent_card = AgentCard(
    name="Flight Agent",
    description="Specialized agent for searching flights.",
    version="1.0.0",
    default_input_modes=["text/plain"],
    default_output_modes=["text/plain"],
    capabilities=AgentCapabilities(streaming=False, extended_agent_card=False),
    supported_interfaces=[
        AgentInterface(
            protocol_binding="JSONRPC",       # which binding we speak
            url="http://127.0.0.1:9999",      # where clients should send messages
            protocol_version="1.0",
        )
    ],
    skills=[skill],
)

# ---------- STEP 4: Request handler (connects protocol <-> executor) ----------
request_handler = DefaultRequestHandler(
    agent_executor=FlightAgentExecutor(),
    task_store=InMemoryTaskStore(),
    agent_card=agent_card,
)

# ---------- STEP 5: Web app (Starlette) with 2 routes ----------
#   GET  /.well-known/agent-card.json  -> returns the Agent Card
#   POST /                             -> receives JSON-RPC messages
app = Starlette(
    routes=[
        *create_agent_card_routes(agent_card),
        *create_jsonrpc_routes(request_handler, "/"),
    ]
)
```

```python
if __name__ == "__main__":
    uvicorn.run(app, host="127.0.0.1", port=9999)
```

ASAAN ALFAAZ MEIN

`execute()` tab chalta hai jab koi message aata hai. Hum `context.message` se text nikalte hain, apna kaam karte hain, aur jawab `event_queue` mein daal dete hain — A2A khud jawab client tak pohncha deta hai.

Step 1 — Start the server

Terminal 1

```
cd class_01
python flight_agent.py
```

Expected output

```
INFO:     Started server process
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:9999 (Press CTRL+C to quit)
```

Step 2 — See the Agent Card in your browser

Open `http://127.0.0.1:9999/.well-known/agent-card.json`. This is the **discovery** URL. Any A2A client reads this first to learn the agent's name, endpoint, binding and skills.

```
{
  "name": "Flight Agent",
  "description": "Specialized agent for searching flights.",
  "supportedInterfaces": [
    {
      "url": "http://127.0.0.1:9999",
      "protocolBinding": "JSONRPC",
      "protocolVersion": "1.0"
    }
  ],
  "version": "1.0.0",
  "capabilities": {
    "streaming": false,
    "extendedAgentCard": false
  },
  "defaultInputModes": [
    "text/plain"
  ],
  "defaultOutputModes": [
    "text/plain"
  ],
  "skills": [
    {
      "id": "search_flights",
      "name": "Search Flights",
      "description": "Searches flight options between cities.",
      "tags": [
        "travel",
        "flight"
      ],
      "examples": [
        "Find flights from Bangalore to Dubai"
      ],
      "inputModes": [
        "text/plain"
      ],
      "outputModes": [
        "text/plain"
      ]
    }
  ]
}
```


Step 3 — Read the card with Python

FOR CLASS_01 `class_01/check_card.py` — Run in Terminal 2

```python
# ============================================================
# CLASS 01 - check_card.py
# Read the Flight Agent's Agent Card (plain HTTP GET, no A2A SDK)
# Run (2nd terminal):  python check_card.py
# ============================================================
import json

import httpx

URL = "http://127.0.0.1:9999/.well-known/agent-card.json"

card = httpx.get(URL).json()

print("Agent name   :", card["name"])
print("Description  :", card["description"])
```

```
print("Endpoint     :", card["supportedInterfaces"][0]["url"])
print("Binding      :", card["supportedInterfaces"][0]["protocolBinding"])
print("Skills       :")
for s in card["skills"]:
    print(f"  - {s['name']}  (id={s['id']})  e.g. {s['examples'][0]}")

print("\nFull card JSON:")
print(json.dumps(card, indent=2, ensure_ascii=False))
```

Terminal 2

```
cd class_01
python check_card.py
```

Output

```
Agent name    : Flight Agent
Description   : Specialized agent for searching flights.
Endpoint      : http://127.0.0.1:9999
Binding       : JSONRPC
Skills        :
  - Search Flights  (id=search_flights)  e.g. Find flights from Bangalore to Dubai
```

Step 4 — Send a raw JSON-RPC message (no A2A client)

This shows exactly what the A2A client does under the hood: an HTTP POST with a JSON-RPC body where the method is `SendMessage`.

FOR CLASS_01 `class_01/raw_jsonrpc_call.py` Run in Terminal 2

```
# ============================================================
# CLASS 01 - raw_jsonrpc_call.py
# Talk to the Flight Agent WITHOUT the A2A client.
# This shows what really travels on the wire: a JSON-RPC message.
# Run (2nd terminal):  python raw_jsonrpc_call.py
# ============================================================
import json
import uuid

import httpx

payload = {
    "jsonrpc": "2.0",
    "id": 1,
    "method": "SendMessage",            # <- the operation name lives INSIDE the JSON
    "params": {
        "message": {
            "messageId": str(uuid.uuid4()),
            "role": "ROLE_USER",
            "parts": [{"text": "Find flights from Bangalore to Dubai"}],
        }
    },
}

response = httpx.post(
    "http://127.0.0.1:9999/",
    json=payload,
    headers={"A2A-Version": "1.0"},     # without this header the server says: version '0.3' not
supported
)
```

```python
data = response.json()
print("RAW RESPONSE:")
print(json.dumps(data, indent=2, ensure_ascii=False))

print("\nJUST THE TEXT:")
print(data["result"]["message"]["parts"][0]["text"])
```

Terminal 2

```
python raw_jsonrpc_call.py
```

Output

```
RAW RESPONSE:
{
  "result": {
    "message": {
      "messageId": "9cb1fe62-829e-4a9a-8580-2dcf63bd059c",
      "role": "ROLE_AGENT",
      "parts": [
        {
          "text": "Flight Agent result:\nBangalore -> Dubai\n1. Emirates EK569 | Non-stop | ₹28,000\n2.
Air India AI995 | 1 stop | ₹24,500\n\nReceived task: Find flights from Bangalore to Dubai"
        }
      ]
    }
  },
  "id": 1,
  "jsonrpc": "2.0"
}

JUST THE TEXT:
Flight Agent result:
Bangalore -> Dubai
1. Emirates EK569 | Non-stop | ₹28,000
2. Air India AI995 | 1 stop | ₹24,500

Received task: Find flights from Bangalore to Dubai
```

GOTCHA FOUND WHILE TESTING

If you leave out the header `A2A-Version: 1.0`, the server rejects the call with *"A2A version '0.3' is not supported by this handler. Expected version '1.0'."* The official A2A client adds this header for you.

PRACTICE — CLASS 01

1. Change the skill's `examples` and `tags`, restart the server and refresh the card URL. What changed?
2. Add a second `AgentSkill` (e.g. `compare_fares`) and put both in `skills=[...]`.
3. Make `execute()` reply differently when the query contains "cheap".
4. Run the server on port 9998 — remember to change **both** the `AgentInterface` URL and `uvicorn.run`.

PRACTICAL CLASS 02

Your first A2A Client

Discover the Flight Agent, check its skills and talk to it with the official client.

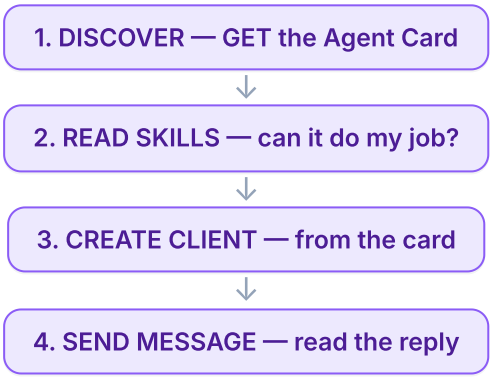
Goal

- Discover an agent from its URL only
- Check that it has the skill you need
- Send several messages and print clean replies
- Handle an agent that is offline

Concepts

- A2A Client, Agent Discovery
- A2ACardResolver, create_client, ClientConfig
- new_text_message, SendMessageRequest
- StreamResponse (message vs task)

The client's 4 steps



FOR CLASS_02 **class_02/flight_client.py** Client

```python
# ============================================================
# CLASS 02 - flight_client.py
# Your first A2A CLIENT: discover the Flight Agent and send it messages
# Needs: class_01/flight_agent.py running in another terminal
# Run:  python flight_client.py
# ============================================================
import asyncio

import httpx

from a2a.client import A2ACardResolver, ClientConfig, create_client
from a2a.helpers import get_message_text, new_text_message
from a2a.types import Role, SendMessageRequest

FLIGHT_AGENT_URL = "http://127.0.0.1:9999"


async def ask_flight_agent(questions: list[str]) -> None:
    async with httpx.AsyncClient() as httpx_client:

        # ---- STEP 1: DISCOVERY - download the Agent Card ----
        resolver = A2ACardResolver(httpx_client=httpx_client, base_url=FLIGHT_AGENT_URL)
        agent_card = await resolver.get_agent_card()
```

```
        print(f"Discovered Agent : {agent_card.name}")
        print(f"Description      : {agent_card.description}")

        # ---- STEP 2: READ SKILLS - can this agent do our job? ----
        skill_ids = [s.id for s in agent_card.skills]
        print("Skills           :", [s.name for s in agent_card.skills])
        if "search_flights" not in skill_ids:
            print("This agent cannot search flights. Stopping.")
            return

        # ---- STEP 3: CREATE A2A CLIENT from the card ----
        # The client reads the card and picks the right binding (JSON-RPC here).
        client = await create_client(
            agent=agent_card,
            client_config=ClientConfig(streaming=False),
        )

        try:
            # ---- STEP 4: SEND MESSAGE(S) ----
            for q in questions:
                message = new_text_message(q, role=Role.ROLE_USER)
                request = SendMessageRequest(message=message)

                print("\n>>> YOU  :", q)
                async for response in client.send_message(request):
                    # The SDK returns a StreamResponse wrapper.
                    # Our agent replies with a direct Message (no Task).
                    if response.HasField("message"):
                        print("<<< AGENT:", get_message_text(response.message))
                    elif response.HasField("task"):
                        print("<<< AGENT created a Task:", response.task.id)
        finally:
            await client.close()


if __name__ == "__main__":
    asyncio.run(
        ask_flight_agent(
            [
                "Find good flights from Bangalore to Dubai.",
                "Cheapest flight from Lahore to Dubai next Friday?",
            ]
        )
    )
```

ASAAN ALFAAZ MEIN

Client ko sirf agent ka **URL** chahiye. Card parh kar woh khud samajh leta hai ke kis endpoint par aur kis binding (JSON-RPC) mein baat karni hai. Skill check karna achhi aadat hai — galat agent ko kaam mat bhejo.

Run it

Terminal 1 — keep the server from Class 01 running

```
cd class_01
python flight_agent.py
```

Terminal 2

```
cd class_02
python flight_client.py
```

Output

```
Discovered Agent : Flight Agent
Description      : Specialized agent for searching flights.
Skills           : ['Search Flights']

>>> YOU  : Find good flights from Bangalore to Dubai.
<<< AGENT: Flight Agent result:
Bangalore -> Dubai
1. Emirates EK569 | Non-stop | ₹28,000
2. Air India AI995 | 1 stop | ₹24,500

Received task: Find good flights from Bangalore to Dubai.

>>> YOU  : Cheapest flight from Lahore to Dubai next Friday?
<<< AGENT: Flight Agent result:
Bangalore -> Dubai
1. Emirates EK569 | Non-stop | ₹28,000
2. Air India AI995 | 1 stop | ₹24,500

Received task: Cheapest flight from Lahore to Dubai next Friday?
```

WHY HASFIELD("MESSAGE")?

`client.send_message()` yields a **StreamResponse** wrapper. It holds *one* of: `message`, `task`, `status_update`, `artifact_update`. Our agent sends a direct Message, so we read `response.message`. If an agent created a Task instead, `response.task` would be filled.

What if the agent is offline?

FOR CLASS_02 `class_02/server_down_test.py` — Stop the server first

```python
# ============================================================
# CLASS 02 - server_down_test.py
# What happens if the remote agent is NOT running?
# Stop flight_agent.py first (Ctrl + C), then run:  python server_down_test.py
# ============================================================
import asyncio

import httpx

from a2a.client import A2ACardResolver


async def main():
    async with httpx.AsyncClient() as httpx_client:
        resolver = A2ACardResolver(httpx_client=httpx_client, base_url="http://127.0.0.1:9999")
        try:
            card = await resolver.get_agent_card()
            print("Agent is UP:", card.name)
        except Exception as e:
            # Discovery fails -> the orchestrator must handle this gracefully
            print("Agent is DOWN. Error type:", type(e).__name__)
            print("Lesson: always wrap remote agent calls in try/except.")
```

```
asyncio.run(main())
```

Terminal 2

```
REM Press Ctrl + C in Terminal 1 to stop flight_agent.py, then:
python server_down_test.py
```

Output

```
Agent is DOWN. Error type: AgentCardResolutionError
Lesson: always wrap remote agent calls in try/except.
```

PRACTICE — CLASS 02

1. Add 3 more questions to the list and check that each reply echoes your question in “Received task”.
2. Change the skill check to look for `find_hotels`. What does the client print now?
3. Print `agent_card.supported_interfaces[0].url` and `.protocol_binding` after discovery.
4. Ask the user for the question with `input()` in a loop and stop on “exit” — a tiny chat with a remote agent.

PRACTICAL CLASS 03

Multi-Agent Travel Orchestrator

One orchestrator delegates sub-tasks to the Flight, Hotel and Visa agents — the full A2A picture.

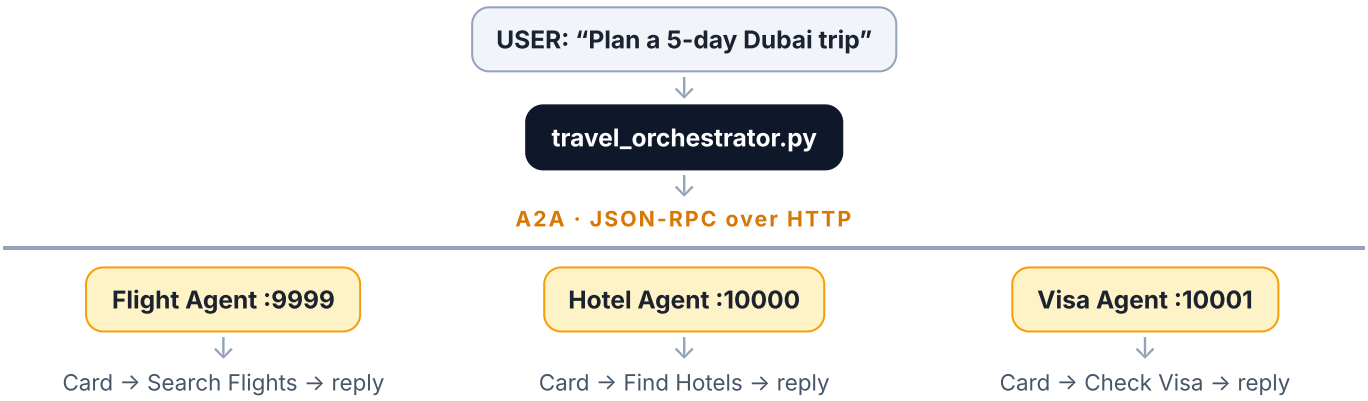
Goal

- Run 3 independent A2A servers on 3 ports
- Build an orchestrator that discovers and calls all of them
- Fix the output bug from the class demo
- Call agents in parallel and survive an agent being down

Concepts

- Orchestrator = A2A client to many servers
- Task delegation
- `asyncio.gather` for parallel calls
- Graceful failure with `try/except`

Architecture (from the board)



The Hotel Agent

Exactly the same pattern as the Flight Agent. Only the executor text, skill, card and port change.

FOR CLASS_03 `class_03/hotel_agent.py` — Server · port 10000

```python
# ============================================================
# CLASS 03 - hotel_agent.py
# Second A2A SERVER: the Hotel Agent (same pattern as the Flight Agent)
# Run:  python hotel_agent.py      (port 10000)
# ============================================================
import uvicorn

from a2a.helpers import get_message_text, new_text_message
from a2a.server.agent_execution import AgentExecutor, RequestContext
from a2a.server.events import EventQueue
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.routes import create_agent_card_routes, create_jsonrpc_routes
from a2a.server.tasks import InMemoryTaskStore
from a2a.types import AgentCapabilities, AgentCard, AgentInterface, AgentSkill
from starlette.applications import Starlette


class HotelAgentExecutor(AgentExecutor):
    async def execute(self, context: RequestContext, event_queue: EventQueue) -> None:
```

```
        query = get_message_text(context.message) if context.message else ""

        result = (
            "Hotel Agent result:\n"
            "Dubai, 5 nights\n"
            "1. Business Bay Hotel | 4-star | ₹9,500/night\n"
            "2. Downtown Executive Hotel | 4-star | ₹11,000/night\n"
            f"\nReceived task: {query}"
        )
        await event_queue.enqueue_event(new_text_message(result))

    async def cancel(self, context: RequestContext, event_queue: EventQueue) -> None:
        raise NotImplementedError("Cancellation is not supported.")


# Agent Skill - what the HOTEL agent can do
skill = AgentSkill(
    id="find_hotels",
    name="Find Hotels",
    description="Finds hotel options for a destination.",
    input_modes=["text/plain"],
    output_modes=["text/plain"],
    tags=["travel", "hotel"],
    examples=["Find a 4-star hotel in Dubai"],
)

# Agent Card - information about the HOTEL agent
agent_card = AgentCard(
    name="Hotel Agent",
    description="Specialized agent for finding hotels.",
    version="1.0.0",
    default_input_modes=["text/plain"],
    default_output_modes=["text/plain"],
    capabilities=AgentCapabilities(streaming=False, extended_agent_card=False),
    supported_interfaces=[
        AgentInterface(
            protocol_binding="JSONRPC",
            url="http://127.0.0.1:10000",
            protocol_version="1.0",
        )
    ],
    skills=[skill],
)

request_handler = DefaultRequestHandler(
    agent_executor=HotelAgentExecutor(),
    task_store=InMemoryTaskStore(),
    agent_card=agent_card,
)

app = Starlette(
    routes=[
        *create_agent_card_routes(agent_card),
        *create_jsonrpc_routes(request_handler, "/"),
    ]
)

if __name__ == "__main__":
    uvicorn.run(app, host="127.0.0.1", port=10000)
```

Challenge solution — the Visa Agent

The board showed a third agent: **Visa Agent**. Here it is, on port 10001.

FOR CLASS_03 **class_03/visa_agent.py** Server · port 10001

```python
# ============================================================
# CLASS 03 - visa_agent.py   (CHALLENGE SOLUTION)
# Third A2A SERVER: the Visa Agent (port 10001)
# Only 4 things change vs. hotel_agent.py: executor text, skill, card, port.
# Run:  python visa_agent.py
# ============================================================
import uvicorn

from a2a.helpers import get_message_text, new_text_message
from a2a.server.agent_execution import AgentExecutor, RequestContext
from a2a.server.events import EventQueue
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.routes import create_agent_card_routes, create_jsonrpc_routes
from a2a.server.tasks import InMemoryTaskStore
from a2a.types import AgentCapabilities, AgentCard, AgentInterface, AgentSkill
from starlette.applications import Starlette

PORT = 10001


class VisaAgentExecutor(AgentExecutor):
    async def execute(self, context: RequestContext, event_queue: EventQueue) -> None:
        query = get_message_text(context.message) if context.message else ""
        result = (
            "Visa Agent result:\n"
            "UAE tourist visa (30 days)\n"
            "Documents: passport (6+ months valid), photo, return ticket, hotel booking\n"
            "Processing time: 3-5 working days (dummy data)\n"
            f"\nReceived task: {query}"
        )
        await event_queue.enqueue_event(new_text_message(result))

    async def cancel(self, context: RequestContext, event_queue: EventQueue) -> None:
        raise NotImplementedError("Cancellation is not supported.")


skill = AgentSkill(
    id="check_visa",
    name="Check Visa Requirements",
    description="Explains visa requirements for a destination.",
    input_modes=["text/plain"],
    output_modes=["text/plain"],
    tags=["travel", "visa"],
    examples=["What visa do I need for Dubai?"],
)

agent_card = AgentCard(
    name="Visa Agent",
    description="Specialized agent for visa requirements.",
    version="1.0.0",
    default_input_modes=["text/plain"],
    default_output_modes=["text/plain"],
    capabilities=AgentCapabilities(streaming=False, extended_agent_card=False),
    supported_interfaces=[
        AgentInterface(
            protocol_binding="JSONRPC",
            url=f"http://127.0.0.1:{PORT}",
            protocol_version="1.0",
        )
    ],
    skills=[skill],
)
```

```
request_handler = DefaultRequestHandler(
    agent_executor=VisaAgentExecutor(),
    task_store=InMemoryTaskStore(),
    agent_card=agent_card,
)

app = Starlette(
    routes=[
        *create_agent_card_routes(agent_card),
        *create_jsonrpc_routes(request_handler, "/"),
    ]
)

if __name__ == "__main__":
    uvicorn.run(app, host="127.0.0.1", port=PORT)
```

The bug in the class demo — and the fix

The original orchestrator checked `isinstance(response, Message)`. With a2a-sdk 1.x that check is always False, because the client yields a **StreamResponse**. So it fell through to `str(response)` and printed a raw protobuf blob:

Original output (messy)

```
================================================================
FLIGHT AGENT RESPONSE
================================================================
message {
  message_id: "f536a3ae-0d22-4c49-9500-15950668bc73"
  role: ROLE_AGENT
  parts {
    text: "Flight Agent result:\nBangalore -> Dubai\n1. Emirates EK569 ..."
  }
}
```

```
# BEFORE (class demo)
async for response in
client.send_message(request):
    if isinstance(response, Message):   # never
True
        return get_message_text(response)
    return str(response)                # raw
blob
```

```
# AFTER (class_03)
async for response in
client.send_message(request):
    if response.HasField("message"):
        return
get_message_text(response.message)
    return str(response)
```

The orchestrator

FOR CLASS_03 **class_03/travel_orchestrator.py** Client to 3 agents

```
# ============================================================
# CLASS 03 - travel_orchestrator.py
# The ORCHESTRATOR (A2A client) that delegates work to many agents.
# Needs running: flight_agent.py (9999), hotel_agent.py (10000)
#                and optionally visa_agent.py (10001)
# Run:  python travel_orchestrator.py
#
# Improvements over the class demo:
#   1. Clean text output  (response.HasField("message") instead of isinstance)
```

```
#   2. try/except         (one agent down does not crash the whole plan)
#   3. Parallel calls     (asyncio.gather -> all agents work at the same time)
# ============================================================
import asyncio
import time

import httpx

from a2a.client import A2ACardResolver, ClientConfig, create_client
from a2a.helpers import get_message_text, new_text_message
from a2a.types import Role, SendMessageRequest

# Which agent gets which sub-task
DELEGATION_PLAN = [
    ("http://127.0.0.1:9999",  "Find good flights from Bangalore to Dubai."),
    ("http://127.0.0.1:10000", "Find a 4-star hotel in Dubai for 5 nights."),
    ("http://127.0.0.1:10001", "What visa does an Indian citizen need for Dubai?"),
]


async def call_remote_agent(base_url: str, task_text: str) -> tuple[str, str]:
    """Discover an agent, send it one message, return (agent_name, reply_text)."""
    try:
        async with httpx.AsyncClient() as httpx_client:
            # 1) Discovery: GET the Agent Card
            resolver = A2ACardResolver(httpx_client=httpx_client, base_url=base_url)
            agent_card = await resolver.get_agent_card()
            print(f"Discovered: {agent_card.name:<12} skills={[s.name for s in agent_card.skills]}")

            # 2) Create a client from the card
            client = await create_client(
                agent=agent_card,
                client_config=ClientConfig(streaming=False),
            )
            try:
                # 3) SendMessage
                request = SendMessageRequest(
                    message=new_text_message(task_text, role=Role.ROLE_USER)
                )
                async for response in client.send_message(request):
                    # FIX: the SDK yields a StreamResponse wrapper, not a Message.
                    if response.HasField("message"):
                        return agent_card.name, get_message_text(response.message)
                    return agent_card.name, str(response)
            finally:
                await client.close()

        return agent_card.name, "No response received."

    except Exception as e:
        return base_url, f"[SKIPPED] Agent not reachable ({type(e).__name__})"


async def main():
    print("=" * 65)
    print("TRAVEL ORCHESTRATOR AGENT")
    print("=" * 65)
    print("USER REQUEST: Plan a 5-day Dubai trip from Bangalore. "
          "I need a flight, a 4-star hotel and visa info.\n")

    start = time.perf_counter()

    # All agents are called AT THE SAME TIME (parallel delegation)
    results = await asyncio.gather(
        *(call_remote_agent(url, task) for url, task in DELEGATION_PLAN)
```

```
    )

    for agent_name, reply in results:
        print("\n" + "=" * 65)
        print(f"{agent_name.upper()} RESPONSE")
        print("=" * 65)
        print(reply)

    print("\n" + "=" * 65)
    print("FINAL ORCHESTRATED RESULT")
    print("=" * 65)
    print(f"Delegated to {len(DELEGATION_PLAN)} independent A2A agents "
          f"in {time.perf_counter() - start:.2f} seconds.")


if __name__ == "__main__":
    asyncio.run(main())
```

ASAAN ALFAAZ MEIN

Orchestrator khud koi flight ya hotel nahi dhoondta. Woh sirf **kaam baant-ta** hai (delegate karta hai), har agent ka card parhta hai, message bhejta hai aur sab jawab jama karke user ko dikhata hai. `asyncio.gather` se teeno agents **ek saath** kaam karte hain.

Run it — 4 terminals

Activate the venv in each terminal first

```
REM Terminal 1
cd class_01 && python flight_agent.py

REM Terminal 2
cd class_03 && python hotel_agent.py

REM Terminal 3
cd class_03 && python visa_agent.py

REM Terminal 4
cd class_03 && python travel_orchestrator.py
```

Output — all 3 agents running

```
======================================================================
TRAVEL ORCHESTRATOR AGENT
======================================================================
USER REQUEST: Plan a 5-day Dubai trip from Bangalore. I need a flight, a 4-star hotel and visa info.

Discovered: Hotel Agent  skills=['Find Hotels']
Discovered: Visa Agent   skills=['Check Visa Requirements']
Discovered: Flight Agent skills=['Search Flights']

======================================================================
FLIGHT AGENT RESPONSE
======================================================================
Flight Agent result:
Bangalore -> Dubai
1. Emirates EK569 | Non-stop | ₹28,000
2. Air India AI995 | 1 stop | ₹24,500

Received task: Find good flights from Bangalore to Dubai.
```

```
================================================================
HOTEL AGENT RESPONSE
================================================================
Hotel Agent result:
Dubai, 5 nights
1. Business Bay Hotel | 4-star | ₹9,500/night
2. Downtown Executive Hotel | 4-star | ₹11,000/night

Received task: Find a 4-star hotel in Dubai for 5 nights.

================================================================
VISA AGENT RESPONSE
================================================================
Visa Agent result:
UAE tourist visa (30 days)
Documents: passport (6+ months valid), photo, return ticket, hotel booking
Processing time: 3-5 working days (dummy data)

Received task: What visa does an Indian citizen need for Dubai?

================================================================
FINAL ORCHESTRATED RESULT
================================================================
Delegated to 3 independent A2A agents in 0.13 seconds.
```

And with the Visa Agent stopped

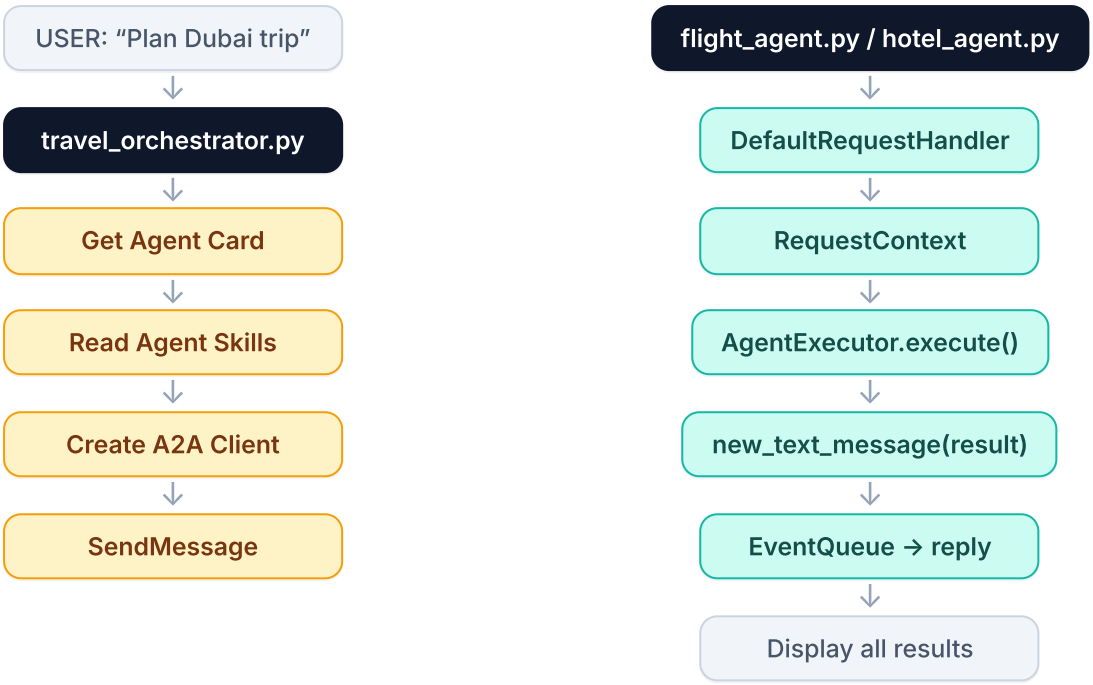
The plan still completes — the missing agent is skipped instead of crashing everything:

Output (end)

```
...
================================================================
HTTP://127.0.0.1:10001 RESPONSE
================================================================
[SKIPPED] Agent not reachable (AgentCardResolutionError)

================================================================
FINAL ORCHESTRATED RESULT
================================================================
Delegated to 3 independent A2A agents in 0.11 seconds.
```

Complete end-to-end flow



PRACTICE — CLASS 03

1. Swap `asyncio.gather` for a normal `for` loop with `await`. Compare the printed time.
2. Add a 4th agent (e.g. **Weather Agent** on port 10002) and add one line to `DELEGATION_PLAN`.
3. Make the orchestrator pick agents by **skill id** instead of hard-coded URLs: discover all cards first, then route each sub-task to the agent whose skill matches.
4. Inside one agent's `execute()`, call an LLM instead of returning dummy text — A2A does not change at all.
5. Bonus: give one agent an MCP tool (e.g. a flight API via MCP) to build the full A2A + MCP architecture from Part A.

PART C

Revision & Interview Q&A

Answers based on the doubt session at the end of the class.

C1 Interview Q&A

Q. If MCP already gives us results, why do we need A2A?

They serve different purposes. **MCP standardizes context** (agent → tools/data). **A2A standardizes communication between agents** (agent → agent). A2A is about the **design** of the system, not the output — you can get the same result without it.

Q. Is A2A only for agents inside the same application?

No. Start by thinking of agents in one system talking to each other, but A2A really shines **across applications and servers** — e.g. your agent calling an agent hosted by another team or company.

Q. Which is used more in industry, MCP or A2A?

MCP. Many companies centralize their tools and resources on MCP servers so any AI app can use them. A2A is optional and used when you have separate multi-agent services.

Q. LangGraph or A2A — which one should I use?

Wrong comparison. **LangGraph is a framework** to build agents and workflows. **A2A is a communication protocol**. Build agents with LangGraph (or any framework) and put A2A between them when they need to talk in a standard way.

Q. Do I need Google ADK to use A2A?

No. Install the **A2A SDK** (`a2a-sdk`).

Q. What are the 3 A2A protocol bindings?

JSON-RPC, **gRPC** and **HTTP+JSON**. Custom bindings like WebSocket or MQTT are also possible.

Q. What does RPC stand for?

Remote Procedure Call — calling a function that runs on another machine.

Q. What is an Agent Card?

A JSON "business card" served at `/.well-known/agent-card.json` with the agent's name, description, endpoint, binding, capabilities and skills. Clients read it to **discover** the agent. It is a schema — not a credit card!

Q. STDIO vs HTTP for MCP servers?

STDIO = same machine, good for practice. HTTP = over a network (local or cloud URL), what real deployments use.

Q. How do you secure an exposed MCP server?

Add an authentication layer: an **API key**, a **JWT bearer token**, or **OAuth 2.1** — and handle token validity and expiry.

Q. How can MCP and FastAPI work together?

Create tools with FastMCP, then put a FastAPI layer in front for HTTP routes, authentication and routing; the client talks to FastAPI.

C2 One-page cheat sheet

A2A connects agents to agents. MCP connects agents to tools and data.

A2A
Agent-to-Agent Protocol · Google · 2025 · framework-independent

MCP
Model Context Protocol · Anthropic · 2024 · tools, resources, prompts

Client
Sends the request (Travel Orchestrator)

Server
Receives the request and does the work (Flight Agent)

Agent Card URL
`GET /.well-known/agent-card.json`

Message endpoint (JSON-RPC)
`POST /` with `"method": "SendMessage"` + header `A2A-Version: 1.0`

Server recipe
AgentExecutor → AgentSkill → AgentCard → DefaultRequestHandler → Starlette + uvicorn

Client recipe
A2ACardResolver → check skills → create_client → send_message → response.message

Bindings
JSON-RPC · gRPC (Protocol Buffers, HTTP/2) · HTTP+JSON

Message / Task / Artifact
Instruction or reply / trackable job with ID & status / final output

Task lifecycle
Submitted → Working → Completed / Failed

Same app?
Just `agent.invoke()` (handoff). Different servers? Use A2A.

Ports used
Flight 9999 · Hotel 10000 · Visa 10001

Install
`pip install a2a-sdk uvicorn starlette httpx`

COMING NEXT IN THE COURSE

Class 57: **Evaluation**, then **Guardrails & Security** — after that, the end-to-end project where MCP and A2A are used together and deployed.